

Shift Left

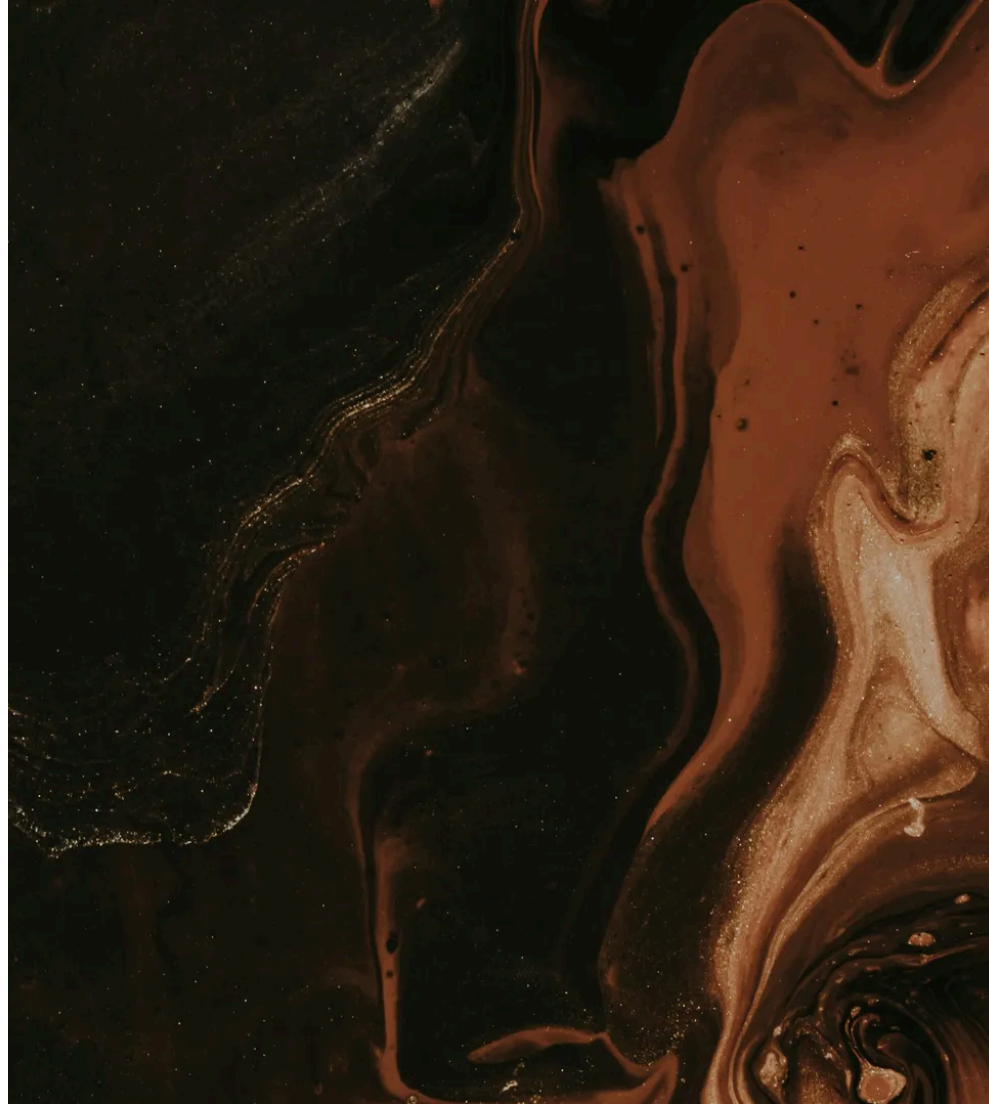
Unit test your compose UI with reusable robots and BDD-style DSL



Iulia STANA
Senior Android Developer
Dawn Technology

Confession

- ✗ Ownership
- ✗ Original
- ✗ Panacea



Confession

- ✗ Ownership
- ✗ Original
- ✗ Panacea

- Compose
- JUnit
- Reusable



Why?

Project constraints: CI build time, (defunct) firebase test lab access, cost

How far can I go with just unit testing?

Unit testing vs. Instrumentation testing

- **Feedback:** fast vs. slow
- **Fidelity:** low vs. high
- **Environment:** local vs. device
- **Clock:** autoAdvance vs. real world

Unit testing vs. Instrumentation testing

- **Feedback:** fast vs. slow
- **Fidelity:** low vs. high
- **Environment:** local vs. device
- **Clock:** autoAdvance vs. real world

Robolectric on JVM 17+ or SDK>30

```
1  android {  
2      testOptions {  
3          unitTests.all {  
4              it.jvmArgs(  
5                  "--add-opens=java.base/java.lang=ALL-UNNAMED",  
6                  //...  
7              )  
8          }  
9      }  
10 }
```

- **Instructions:** <https://robolectric.org/getting-started/#running-with-java-17-and-higher>

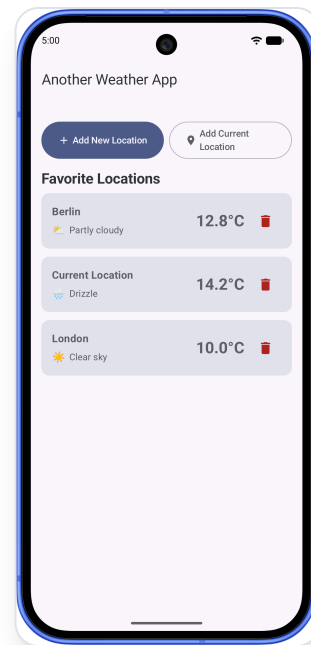
Robolectric on JVM 17+ or SDK>30

```
1  android {  
2      testOptions {  
3          unitTests.all {  
4              it.jvmArgs(  
5                  "--add-opens=java.base/java.lang=ALL-UNNAMED",  
6                  //...  
7              )  
8          }  
9      }  
10 }
```

- **Instructions:** <https://robolectric.org/getting-started/#running-with-java-17-and-higher>

Toy Weather App

Small weather app that shows favorites locations, can add/remove them and the weather details for a given location.



Permission Flows

Don't check the system! Tests cannot interact with system prompts.

Rule: Don't try to verify that a prompt is *shown* or *clicked*

Verify that it was **triggered** and control the **result**.

For direct grant/deny:

```
1  val application = ApplicationProvider.getApplicationContext<Application>()
2  val shadowApp = shadowOf(application)
3
4  shadowApp.grantPermissions(Manifest.permission.CAMERA)
5  shadowApp.denyPermissions(Manifest.permission.ACCESS_FINE_LOCATION)
```

Permission Flows

Don't check the system! Tests cannot interact with system prompts.

Rule: Don't try to verify that a prompt is *shown* or *clicked*

Verify that it was **triggered** and control the **result**.

For direct grant/deny:

```
1  val application = ApplicationProvider.getApplicationContext<Application>()
2  val shadowApp = shadowOf(application)
3
4  shadowApp.grantPermissions(Manifest.permission.CAMERA)
5  shadowApp.denyPermissions(Manifest.permission.ACCESS_FINE_LOCATION)
```

Permission Flows

Don't check the system! Tests cannot interact with system prompts.

Rule: Don't try to verify that a prompt is *shown* or *clicked*

Verify that it was **triggered** and control the **result**.

For direct grant/deny:

```
1  val application = ApplicationProvider.getApplicationContext<Application>()
2  val shadowApp = shadowOf(application)
3
4  shadowApp.grantPermissions(Manifest.permission.CAMERA)
5  shadowApp.denyPermissions(Manifest.permission.ACCESS_FINE_LOCATION)
```

Create our own **ActivityResultRegistry**

Receive the permission requests, return the results

```
1  class PermissionPromptRegistry : ActivityResultRegistry(),
2      ActivityResultRegistryOwner {
3      override fun getActivityResultRegistry() = this
4
5      override fun <I, O> onLaunch(
6          requestCode: Int,
7          contract: ActivityResultContract<I, O>,
8          input: I,
9          options: ActivityOptionsCompat?
10     ) {
11         //process permission request
12         dispatchResult(requestCode, result)
13     }
14 }
```

Create our own **ActivityResultRegistry**

Receive the permission requests, return the results

```
1  class PermissionPromptRegistry : ActivityResultRegistry(),
2      ActivityResultRegistryOwner {
3      override fun getActivityResultRegistry() = this
4
5      override fun <I, O> onLaunch(
6          requestCode: Int,
7          contract: ActivityResultContract<I, O>,
8          input: I,
9          options: ActivityOptionsCompat?
10     ) {
11         //process permission request
12         dispatchResult(requestCode, result)
13     }
14 }
```

Create our own **ActivityResultRegistry**

Receive the permission requests, return the results

```
1  class PermissionPromptRegistry : ActivityResultRegistry(),
2      ActivityResultRegistryOwner {
3      override fun getActivityResultRegistry() = this
4
5      override fun <I, O> onLaunch(
6          requestCode: Int,
7          contract: ActivityResultContract<I, O>,
8          input: I,
9          options: ActivityOptionsCompat?
10     ) {
11         //process permission request
12         dispatchResult(requestCode, result)
13     }
14 }
```

Permission Unit Testing: Setup

```
1  val registry = PermissionPromptRegistry()
2  composeTestRule.setContent {
3      CompositionLocalProvider(
4          LocalActivityResultRegistryOwner provides registry
5      ) {
6          AwaTheme {
7              LocationPermissionScreen(onBackClick = {})
8          }
9      }
10 }
```

Permission Unit Testing: Setup

```
1  val registry = PermissionPromptRegistry()
2  composeTestRule.setContent {
3      CompositionLocalProvider(
4          LocalActivityResultRegistryOwner provides registry
5      ) {
6          AwaTheme {
7              LocationPermissionScreen(onBackClick = {})
8          }
9      }
10 }
```

Screen Unit Testing: Setup

JUnit starting from the `Screen` composable

```
1  @RunWith(RobolectricTestRunner::class)
2  class FavoritesScreenTest {
3      @get:Rule
4      val composeTestRule = createComposeRule()
5
6      @Test
7      fun `given no favorites, screen displays empty state and action buttons`() {
8          var lambdaCalled = false
9          composeTestRule.setContent {
10              AwaTheme {
11                  FavoritesScreen(
12                      favorites = emptyList(),
13                      onAddLocationClick = { lambdaCalled = true },
14                      isLoadingLocation = false,
```

Screen Unit Testing: Setup

JUnit starting from the `Screen` composable

```
1  @RunWith(RobolectricTestRunner::class)
2  class FavoritesScreenTest {
3      @get:Rule
4      val composeTestRule = createComposeRule()
5
6      @Test
7      fun `given no favorites, screen displays empty state and action buttons`() {
8          var lambdaCalled = false
9          composeTestRule.setContent {
10              AwaTheme {
11                  FavoritesScreen(
12                      favorites = emptyList(),
13                      onAddLocationClick = { lambdaCalled = true },
14                      isLoadingLocation = false,
```

Screen Unit Testing: Setup

JUnit starting from the `Screen` composable

```
1  @RunWith(RobolectricTestRunner::class)
2  class FavoritesScreenTest {
3      @get:Rule
4      val composeTestRule = createComposeRule()
5
6      @Test
7      fun `given no favorites, screen displays empty state and action buttons`() {
8          var lambdaCalled = false
9          composeTestRule.setContent {
10              AwaTheme {
11                  FavoritesScreen(
12                      favorites = emptyList(),
13                      onAddLocationClick = { lambdaCalled = true },
14                      isLoadingLocation = false,
```

Screen Unit Testing: Setup

JUnit starting from the `Screen` composable

```
1  @RunWith(RobolectricTestRunner::class)
2  class FavoritesScreenTest {
3      @get:Rule
4      val composeTestRule = createComposeRule()
5
6      @Test
7      fun `given no favorites, screen displays empty state and action buttons`() {
8          var lambdaCalled = false
9          composeTestRule.setContent {
10              AwaTheme {
11                  FavoritesScreen(
12                      favorites = emptyList(),
13                      onAddLocationClick = { lambdaCalled = true },
14                      isLoadingLocation = false,
```

Screen Unit Testing: Setup

JUnit starting from the `Screen` composable

```
1  @RunWith(RobolectricTestRunner::class)
2  class FavoritesScreenTest {
3      @get:Rule
4      val composeTestRule = createComposeRule()
5
6      @Test
7      fun `given no favorites, screen displays empty state and action buttons`() {
8          var lambdaCalled = false
9          composeTestRule.setContent {
10              AwakeTheme {
11                  FavoritesScreen(
12                      favorites = emptyList(),
13                      onAddLocationClick = { lambdaCalled = true },
14                      isLoadingLocation = false,
```

Screen Unit Testing - Verification

What can be verified on a `Screen` with a given state?

```
1  composeTestRule.onNodeWithText("No Favorites Added")
2      .assertIsDisplayed()
3
4  val addButton = composeTestRule.onNodeWithTag("favorite:add_button")
5  addButton.assertIsDisplayed()
6      .assertIsEnabled()
7      .assertHasClickAction()
8      .assert(hasTraversalIndex(1f))
9
10 addButton.performClick()
11 // onAddLocationClick = { lambdaCalled = true },
12 assertTrue(lambdaCalled)
13
```

Screen Unit Testing - Verification

What can be verified on a `Screen` with a given state?

```
1  composeTestRule.onNodeWithText("No Favorites Added")
2      .assertIsDisplayed()
3
4  val addButton = composeTestRule.onNodeWithTag("favorite:add_button")
5  addButton.assertIsDisplayed()
6      .assertIsEnabled()
7      .assertHasClickAction()
8      .assert(hasTraversalIndex(1f))
9
10 addButton.performClick()
11 // onAddLocationClick = { lambdaCalled = true },
12 assertTrue(lambdaCalled)
13
```

Screen Unit Testing - Verification

What can be verified on a `Screen` with a given state?

```
1  composeTestRule.onNodeWithText("No Favorites Added")
2      .assertIsDisplayed()
3
4  val addButton = composeTestRule.onNodeWithTag("favorite:add_button")
5  addButton.assertIsDisplayed()
6      .assertIsEnabled()
7      .assertHasClickAction()
8      .assert(hasTraversalIndex(1f))
9
10 addButton.performClick()
11 // onAddLocationClick = { lambdaCalled = true },
12 assertTrue(lambdaCalled)
13
```

Screen Unit Testing - Verification

What can be verified on a `Screen` with a given state?

```
1  composeTestRule.onNodeWithText("No Favorites Added")
2      .assertIsDisplayed()
3
4  val addButton = composeTestRule.onNodeWithTag("favorite:add_button")
5  addButton.assertIsDisplayed()
6      .assertIsEnabled()
7      .assertHasClickAction()
8      .assert(hasTraversalIndex(1f))
9
10 addButton.performClick()
11 // onAddLocationClick = { lambdaCalled = true },
12 assertTrue(lambdaCalled)
13
```

Screen Unit Testing - Verification

What can be verified on a `Screen` with a given state?

```
1  composeTestRule.onNodeWithText("No Favorites Added")
2      .assertIsDisplayed()
3
4  val addButton = composeTestRule.onNodeWithTag("favorite:add_button")
5  addButton.assertIsDisplayed()
6      .assertIsEnabled()
7      .assertHasClickAction()
8      .assert(hasTraversalIndex(1f))
9
10 addButton.performClick()
11 // onAddLocationClick = { lambdaCalled = true },
12 assertTrue(lambdaCalled)
13
```

Screen Unit Testing: Verdict

- basic state verification, accessibility + traversal index
- one step above screenshot testing
- no transitions or state modification
- lambda call check
- ~ previews
- Litmus state check

Screen Unit Testing: Verdict

- basic state verification, accessibility + traversal index
- one step above screenshot testing
- no transitions or state modification
- lambda call check
- ~ previews
- Litmus state check

UNDERWHELMING!

Screen Unit Testing: Verdict

- basic state verification, accessibility + traversal index
- one step above screenshot testing
- no transitions or state modification
- lambda call check
- ~ previews
- Litmus state check

UNDERWHELMING!

Change the System Under Test?

Move the test higher, at the `Route` composable

```
1  @Composable
2  fun FavoritesRoute(
3      modifier: Modifier = Modifier,
4      viewModel: FavoritesViewModel = hiltViewModel()
5  ) {
6      val favorites by viewModel.favoritesState.collectAsStateWithLifecycle()
7      FavoritesScreen(
8          favorites = favorites,
9          onAddLocationClick = viewModel::onAddLocationClick,
10         onAddCurrentLocationClick = viewModel::addCurrentLocationToFavorites,
11         onLocationClick = viewModel::onLocationClick,
12         onRemoveFavorite = viewModel::removeFavorite,
13         modifier = modifier
```

Route Unit Testing: Setup

More setup required: all `ViewModel` dependencies, but same compose rule

```
1  @Test
2  fun `given location is removed, location is not shown`() = runTest {
3      val viewModel = FavoritesViewModel(
4          favoritesRepository = fakeFavoritesRepo,
5          weatherRepository = fakeWeatherRepo,
6          locationTracker = fakeLocationTracker,
7          navigator = navigator
8      )
9
10     composeTestRule.setContent {
11         AwaTheme {
12             FavoritesRoute(viewModel = viewModel)
13         }
14     }
```

Route Unit Testing: Setup

More setup required: all `ViewModel` dependencies, but same compose rule

```
1  @Test
2  fun `given location is removed, location is not shown`() = runTest {
3      val viewModel = FavoritesViewModel(
4          favoritesRepository = fakeFavoritesRepo,
5          weatherRepository = fakeWeatherRepo,
6          locationTracker = fakeLocationTracker,
7          navigator = navigator
8      )
9
10     composeTestRule.setContent {
11         AwaTheme {
12             FavoritesRoute(viewModel = viewModel)
13         }
14     }
```

Route Unit Testing: Setup

More setup required: all `ViewModel` dependencies, but same compose rule

```
1  @Test
2  fun `given location is removed, location is not shown`() = runTest {
3      val viewModel = FavoritesViewModel(
4          favoritesRepository = fakeFavoritesRepo,
5          weatherRepository = fakeWeatherRepo,
6          locationTracker = fakeLocationTracker,
7          navigator = navigator
8      )
9
10     composeTestRule.setContent {
11         AwaTheme {
12             FavoritesRoute(viewModel = viewModel)
13         }
14     }
```

Route Unit Testing: Verification

Can verify state transition in UI, with real `ViewModel` and fake data sources

```
1  composeTestRule.onNodeWithContentDescription("Remove Favorite")
2      .performClick()
3
4  composeTestRule.onNodeWithText("Berlin").assertDoesNotExist()
5  composeTestRule.onNodeWithText("No Favorites Added").assertIsDisplayed()
```

Route Unit Testing: Verification

Can verify state transition in UI, with real `ViewModel` and fake data sources

```
1  composeTestRule.onNodeWithContentDescription("Remove Favorite")
2      .performClick()
3
4  composeTestRule.onNodeWithText("Berlin").assertDoesNotExist()
5  composeTestRule.onNodeWithText("No Favorites Added").assertIsDisplayed()
```

Route Unit Testing: Verification

Can verify state transition in UI, with real `ViewModel` and fake data sources

```
1 composeTestRule.onNodeWithContentDescription("Remove Favorite")
2   .performClick()
3
4 composeTestRule.onNodeWithText("Berlin").assertDoesNotExist()
5 composeTestRule.onNodeWithText("No Favorites Added").assertIsDisplayed()
```



Route Unit Testing: Verification

Can verify state transition in UI, with real `ViewModel` and fake data sources

```
1 composeTestRule.onNodeWithContentDescription("Remove Favorite")
2     .performClick()
3
4 composeTestRule.onNodeWithText("Berlin").assertDoesNotExist()
5 composeTestRule.onNodeWithText("No Favorites Added").assertIsDisplayed()
```



Route Unit Testing: Summary

- **More** fakes/doubles/mocks involved
 - Maybe Hilt
- **Can cover**
 - state modification
 - some navigation state checks (sans UI)

Change the system under test... again

Move the test even higher, at the composable holding our `NavDisplay`

- Even more fakes
 - Get `Hilt` involved
- Can check against more than one screens
- State transitions
- Navigation state changes
- Navigation with UI

App Unit Testing: Setup

All about Hilt

```
1  @HiltAndroidTest
2  @RunWith(RobolectricTestRunner::class)
3  @Config(application = HiltTestApplication::class)
4  class AwaAppTest {
5
6      @get:Rule(order = 0)
7      val hiltRule = HiltAndroidRule(this)
8
9      @get:Rule(order = 1)
10     val composeTestRule = createAndroidComposeRule<MainActivity>()
11
12     @Before
13     fun setUp() {
14         hiltRule.inject()
```

App Unit Testing: Setup

All about Hilt

```
1  @HiltAndroidTest
2  @RunWith(RobolectricTestRunner::class)
3  @Config(application = HiltTestApplication::class)
4  class AwaAppTest {
5
6      @get:Rule(order = 0)
7      val hiltRule = HiltAndroidRule(this)
8
9      @get:Rule(order = 1)
10     val composeTestRule = createAndroidComposeRule<MainActivity>()
11
12     @Before
13     fun setUp() {
14         hiltRule.inject()
```



App Unit Testing: Setup

Different rule!

```
1  @HiltAndroidTest
2  @RunWith(RobolectricTestRunner::class)
3  @Config(application = HiltTestApplication::class)
4  class AwaAppTest {
5
6      @get:Rule(order = 0)
7      val hiltRule = HiltAndroidRule(this)
8
9      @get:Rule(order = 1)
10     val composeTestRule = createAndroidComposeRule<MainActivity>()
11
12     @Before
13     fun setUp() {
14         hiltRule.inject()
```

App Unit Testing: Summary

- Setup: **everything**
- Can cover: **everything**
 - User flows across multiple screens
 - UI navigation checks
 - Fake data sources

Side quest: Unit Test Lifecycle Side Effects

We can provide our own `LifecycleOwner` for our tests.

```
1  private class TestLifecycleOwner(  
2      initialState: Lifecycle.State = Lifecycle.State.INITIALIZED  
3  ) : LifecycleOwner {  
4      override val lifecycle: Lifecycle  
5          field = LifecycleRegistry(this).apply {  
6              currentState = initialState  
7          }  
8  
9      fun handleLifecycleEvent(event: Lifecycle.Event) {  
10         lifecycle.handleLifecycleEvent(event)  
11     }  
12 }
```

Lifecycle Side Effects Unit Testing: Setup

```
1  val lifecycleOwner =  
2  TestLifecycleOwner(initialState = Lifecycle.State.INITIALIZED)  
3  
4  composeTestRule.setContent {  
5      CompositionLocalProvider(LocalLifecycleOwner provides lifecycleOwner) {  
6          AwaTheme {  
7              FavoritesRoute(viewModel = viewModel)  
8          }  
9      }  
10 }  
11 lifecycleOwner.handleLifecycleEvent(Lifecycle.Event.ON_RESUME)
```

Lifecycle Side Effects Unit Testing: Verification

```
1  composeTestRule.onNodeWithText("21.5°C").assertIsDisplayed()
2
3  fakeWeatherRepo.weatherResult =
4  Result.success(fakeWeather.copy(currentTemperature = 28.0))
5
6  lifecycleOwner.handleLifecycleEvent(Lifecycle.Event.ON_PAUSE)
7  composeTestRule.onNodeWithText("21.5°C").assertIsDisplayed()
8
9  lifecycleOwner.handleLifecycleEvent(Lifecycle.Event.ON_RESUME)
10 composeTestRule.onNodeWithText("28.0°C").assertIsDisplayed()
```

Lifecycle Side Effects Unit Testing: Verification

```
1  composeTestRule.onNodeWithText("21.5°C").assertIsDisplayed()
2
3  fakeWeatherRepo.weatherResult =
4  Result.success(fakeWeather.copy(currentTemperature = 28.0))
5
6  lifecycleOwner.handleLifecycleEvent(Lifecycle.Event.ON_PAUSE)
7  composeTestRule.onNodeWithText("21.5°C").assertIsDisplayed()
8
9  lifecycleOwner.handleLifecycleEvent(Lifecycle.Event.ON_RESUME)
10 composeTestRule.onNodeWithText("28.0°C").assertIsDisplayed()
```

Lifecycle Side Effects Unit Testing: Verification

```
1  composeTestRule.onNodeWithText("21.5°C").assertIsDisplayed()
2
3  fakeWeatherRepo.weatherResult =
4  Result.success(fakeWeather.copy(currentTemperature = 28.0))
5
6  lifecycleOwner.handleLifecycleEvent(Lifecycle.Event.ON_PAUSE)
7  composeTestRule.onNodeWithText("21.5°C").assertIsDisplayed()
8
9  lifecycleOwner.handleLifecycleEvent(Lifecycle.Event.ON_RESUME)
10 composeTestRule.onNodeWithText("28.0°C").assertIsDisplayed()
```

Lifecycle Side Effects Unit Testing: Verification

```
1  composeTestRule.onNodeWithText("21.5°C").assertIsDisplayed()
2
3  fakeWeatherRepo.weatherResult =
4  Result.success(fakeWeather.copy(currentTemperature = 28.0))
5
6  lifecycleOwner.handleLifecycleEvent(Lifecycle.Event.ON_PAUSE)
7  composeTestRule.onNodeWithText("21.5°C").assertIsDisplayed()
8
9  lifecycleOwner.handleLifecycleEvent(Lifecycle.Event.ON_RESUME)
10 composeTestRule.onNodeWithText("28.0°C").assertIsDisplayed()
```

Lifecycle Side Effects Unit Testing: Verification

```
1  composeTestRule.onNodeWithText("21.5°C").assertIsDisplayed()
2
3  fakeWeatherRepo.weatherResult =
4  Result.success(fakeWeather.copy(currentTemperature = 28.0))
5
6  lifecycleOwner.handleLifecycleEvent(Lifecycle.Event.ON_PAUSE)
7  composeTestRule.onNodeWithText("21.5°C").assertIsDisplayed()
8
9  lifecycleOwner.handleLifecycleEvent(Lifecycle.Event.ON_RESUME)
10 composeTestRule.onNodeWithText("28.0°C").assertIsDisplayed()
```

Side Quest: Unit Test Deep Links

Use `ActivityScenario` so that we can:

- **Launch** an Activity in a realistic environment.
- **Drive Lifecycle States** (`CREATED` , `STARTED` , `RESUMED` , `DESTROYED`).
- **Trigger Configuration Changes** (e.g. screen rotation / recreation via `.recreate()`).
- **Safely Access the Activity Instance** on the main thread via `.onActivity { activity
-> ... } .`

Unit Test Deep Links: Setup

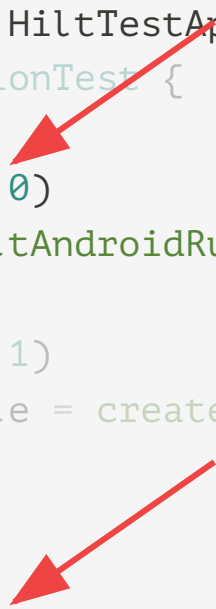
Use `createEmptyComposeRule()`

```
1  @HiltAndroidTest
2  @RunWith(RobolectricTestRunner::class)
3  @Config(application = HiltTestApplication::class)
4  class DeepLinkNavigationTest {
5
6      @get:Rule(order = 0)
7      val hiltRule = HiltAndroidRule(this)
8
9      @get:Rule(order = 1)
10     val composeTestRule = createEmptyComposeRule()
11
12     @Before
13     fun setUp() {
14         hiltRule.inject()
```

Unit Test Deep Links: Setup

Use `createEmptyComposeRule()`

```
1  @HiltAndroidTest
2  @RunWith(RobolectricTestRunner::class)
3  @Config(application = HiltTestApplication::class)
4  class DeepLinkNavigationTest {
5
6      @get:Rule(order = 0)
7      val hiltRule = HiltAndroidRule(this)
8
9      @get:Rule(order = 1)
10     val composeTestRule = createEmptyComposeRule()
11
12     @Before
13     fun setUp() {
14         hiltRule.inject()
```



Unit Test Deep Links: Verification

```
1  val deepLinkIntent = Intent(  
2      Intent.ACTION_VIEW,  
3      Uri.parse("https://org.me.awa/weather?name=Berlin&lat=52.52&lon=13.405")  
4  ).apply {  
5      setPackage(ApplicationProvider.getApplicationContext<Context>().packageName)  
6  }  
7  ActivityScenario.launch<MainActivity>(deepLinkIntent).use {  
8      composeTestRule.onAllNodesWithText("Berlin")[0].assertIsDisplayed()  
9      composeTestRule.onNodeWithText("21.5°C").assertIsDisplayed()  
10 }
```

Unit Test Deep Links: Verification

```
1  val deepLinkIntent = Intent(  
2      Intent.ACTION_VIEW,  
3      Uri.parse("https://org.me.awa/weather?name=Berlin&lat=52.52&lon=13.405")  
4  ).apply {  
5      setPackage(ApplicationProvider.getApplicationContext<Context>().packageName)  
6  }  
7  ActivityScenario.launch<MainActivity>(deepLinkIntent).use {  
8      composeTestRule.onAllNodesWithText("Berlin")[0].assertIsDisplayed()  
9      composeTestRule.onNodeWithText("21.5°C").assertIsDisplayed()  
10 }
```

Unit Test Deep Links: Verification

```
1  val deepLinkIntent = Intent(  
2      Intent.ACTION_VIEW,  
3      Uri.parse("https://org.me.awa/weather?name=Berlin&lat=52.52&lon=13.405")  
4      ).apply {  
5          setPackage(ApplicationProvider.getApplicationContext<Context>().packageName)  
6      }  
7  ActivityScenario.launch<MainActivity>(deepLinkIntent).use {  
8      composeTestRule.onAllNodesWithText("Berlin")[0].assertIsDisplayed()  
9      composeTestRule.onNodeWithText("21.5°C").assertIsDisplayed()  
10 }
```

createComposeRule ()

Feature / Goal	<code>createComposeRule()</code>
Activity Host	Blank internal <code>ComponentActivity</code>
Hilt ViewModel Injection	❌ Fails (Activity lacks <code>@AndroidEntryPoint</code>)
Lifecycle Control	Requires custom <code>TestLifecycleOwner</code>
Custom Deep Link Intent Launch	❌ Not supported
Compose UI Checks (<code>onNodeWithText</code>)	✅ Supported (<code>setContent { ... }</code>)
Best Used For	Isolated Composable & Route unit tests

createAndroidComposeRule ()

Feature / Goal	<code>createAndroidComposeRule<MainActivity>()</code>
Activity Host	Real <code>MainActivity</code>
Hilt ViewModel Injection	✅ Full Hilt Support
Lifecycle Control	✅ <code>composeTestRule.activityRule.scenario</code>
Custom Deep Link Intent Launch	❌ Uses default launch Intent
Compose UI Checks (<code>onNodeWithText</code>)	✅ Supported (Auto-hosted or <code>setContent</code>)
Best Used For	Full App Navigation & Integration unit tests

ActivityScenario.launch (Intent)

Feature / Goal	Direct <code>ActivityScenario.launch(Intent)</code>
Activity Host	Real <code>MainActivity</code>
Hilt ViewModel Injection	✅ Full Hilt Support
Lifecycle Control	✅ <code>scenario.moveToState()</code> , <code>recreate()</code>
Custom Deep Link Intent Launch	✅ Full support (<code>launch(intent)</code>)
Compose UI Checks (<code>onNodeWithText</code>)	✅ Supported via <code>createEmptyComposeRule()</code>
Best Used For	Deep Links, Intents & Activity Lifecycle tests

A Shared Foundation

Wait a minute...

Both JUnit and instrumentation tests rely on the same APIs.

The only difference is the execution environment.

UI evolves

Yeah, but... my UI changes often

UI evolves

Yeah, but... my UI changes often

Could we contain these changes, limit their impact on our tests?

Robot screen

- Define your "system under test" in one single place
- Make sure it's stable
- Self Contained
- One UI change == one correction

Robot Example

`ComposeTestRule` is implemented by both `AndroidComposeTestRule` and `ComposeTestRule`

```
1  class FavoriteRobotScreen(private val rule: ComposeTestRule) {
2      val root: SemanticsNodeInteraction
3          get() = rule.onNodeWithTag("favorite:root")
4
5      val favoritesHeader: SemanticsNodeInteraction
6          get() = rule.onNodeWithText("Favorite Locations")
7
8      fun clickAddNewLocation(): FavoriteRobotScreen {
9          addButton.performClick()
10         return this
11     }
12 }
```

Robot Example

`ComposeTestRule` is implemented by both `AndroidComposeTestRule` and `ComposeTestRule`

```
1  class FavoriteRobotScreen(private val rule: ComposeTestRule) {
2      val root: SemanticsNodeInteraction
3          get() = rule.onNodeWithTag("favorite:root")
4
5      val favoritesHeader: SemanticsNodeInteraction
6          get() = rule.onNodeWithText("Favorite Locations")
7
8      fun clickAddNewLocation(): FavoriteRobotScreen {
9          addButton.performClick()
10         return this
11     }
12 }
```

Robot Example

`ComposeTestRule` is implemented by both `AndroidComposeTestRule` and `ComposeTestRule`

```
1  class FavoriteRobotScreen(private val rule: ComposeTestRule) {
2      val root: SemanticsNodeInteraction
3          get() = rule.onNodeWithTag("favorite:root")
4
5      val favoritesHeader: SemanticsNodeInteraction
6          get() = rule.onNodeWithText("Favorite Locations")
7
8      fun clickAddNewLocation(): FavoriteRobotScreen {
9          addButton.performClick()
10         return this
11     }
12 }
```

Robot Example

`ComposeTestRule` is implemented by both `AndroidComposeTestRule` and `ComposeTestRule`

```
1  class FavoriteRobotScreen(private val rule: ComposeTestRule) {
2      val root: SemanticsNodeInteraction
3          get() = rule.onNodeWithTag("favorite:root")
4
5      val favoritesHeader: SemanticsNodeInteraction
6          get() = rule.onNodeWithText("Favorite Locations")
7
8      fun clickAddNewLocation(): FavoriteRobotScreen {
9          addNewButton.performClick()
10         return this
11     }
12 }
```

Node is MIA

- Beware of **implicit merges**: `Modifier.clickable`
- Intentional accessibility merges
- Unmerge tree to find it:

```
1 fun onNode(  
2     matcher: SemanticsMatcher,  
3     useUnmergedTree: Boolean = false,  
4 ): SemanticsNodeInteraction
```

Node is MIA

- Beware of **implicit merges**: `Modifier.clickable`
- Intentional accessibility merges
- Unmerge tree to find it:

```
1 fun onNode(  
2     matcher: SemanticsMatcher,  
3     useUnmergedTree: Boolean = true,  
4 ): SemanticsNodeInteraction
```

Make It Stable

Fail fast screen readiness guard

```
1  init {
2      rule.waitUntil(3_000) {
3          try {
4              root.assertExists()
5              true
6          } catch (e: AssertionError) {
7              false
8          }
9      }
10 }
```

Make It Reusable

- Single module app: `testFixtures`
- Multi-module app: **dedicated module** reused in `tests` & `androidTest`

```
1  android {  
2      @Suppress("UnstableApiUsage")  
3      testFixtures { enable = true }  
4  }  
5
```

Robot In Use

The checks performed are closer to the actual semantics of the screen

```
1 composeTestRule.onNodeWithTag("favorite:root").assertIsDisplayed()  
2  
3 composeTestRule.onNodeWithText("Berlin").assertIsDisplayed()  
4 composeTestRule.onNodeWithText("Tokyo").assertIsDisplayed()
```

Robot In Use

The checks performed are closer to the actual semantics of the screen

```
1  val robot = FavoriteRobotScreen(composeTestRule)
2  // List items targeted deterministically by unique location id
3  robot.favoriteItem("1").assertIsDisplayed().assertTextContains("Berlin")
4  robot.favoriteItem("2").assertIsDisplayed().assertTextContains("Tokyo")
```

Robot In Use

The checks performed are closer to the actual semantics of the screen

```
1  val robot = FavoriteRobotScreen(composeTestRule)
2  // List items targeted deterministically by unique location id
3  robot.favoriteItem("1").assertIsDisplayed().assertTextContains("Berlin")
4  robot.favoriteItem("2").assertIsDisplayed().assertTextContains("Tokyo")
```



Robot In Use

The checks performed are closer to the actual semantics of the screen

```
1  val robot = FavoriteRobotScreen(composeTestRule)
2  // List items targeted deterministically by unique location id
3  robot.favoriteItem("1").assertIsDisplayed().assertTextContains("Berlin")
4  robot.favoriteItem("2").assertIsDisplayed().assertTextContains("Tokyo")
```



Great... but what is it that we're actually testing?

- How do we know what's automated vs requires manual testing?
- Meaningful coverage information, not just a percentage number

"Diploma" Agile vs Reality

- Cryptic feature description
- Little to no acceptance criteria
- Discovery during development
- Delivered feature, QA asks: how do I test this?
- Circus

Feature files

- Simple, plain old text files
- Natural language
- Given/When/Then format
- Platform agnostic
- Product manual
- Onboarding docs
- Breakdown per feature, each feature 1 or more scenarios

Feature files - but written well

Each individual scenario:

- State teleportation
- Reusable preconditions
- Mutually exclusive
- Independent of one another
- One scenario == one observable behavior

Feature file example

Typical content

Feature: Favorites Screen Behavior

As a user of the weather app

I want to view and manage my saved favorite locations and their current weather

So that I can quickly check weather updates for locations I care about

@Favorites @FA-01 @Priority_1 @Android

Scenario: FA-01 - **Empty state is displayed when no favorite locations are saved**

Preconditions: @Preconditions_Favorites_EmptyState

Given I have no saved favorite locations

When I view the Favorites screen

Then I see the empty state view with title "No Favorites Added"

And I see the subtitle "Tap '+' to search and save your favorite cities."

And I see the "Add Favorite Location" button enabled

And I see the "Add Current Location" button enabled

Feature file example

Typical content

Feature: Favorites Screen Behavior

As a user of the weather app

I want to view and manage my saved favorite locations and their current weather

So that I can quickly check weather updates for locations I care about

@Favorites @FA-01 @Priority_1 @Android

Scenario: FA-01 - Empty state is displayed when no favorite locations are saved

Preconditions: @Preconditions_Favorites_EmptyState

Given I have no saved favorite locations

When I view the Favorites screen

Then I see the empty state view with title "No Favorites Added"

And I see the subtitle "Tap '+' to search and save your favorite cities."

And I see the "Add Favorite Location" button enabled

And I see the "Add Current Location" button enabled

What do we do with it?

Well...

We implement it.

Kotlin Domain Specific Language (DSL)

What is it?

The test DSL is a readable domain bridge between
product specifications (Gherkin)

and

our Compose test code

Kotlin DSL

1. Built-in Flakiness Protection
2. Living Documentation
3. Constant & Continuous Language
4. Standardized Assertion Boundary
5. Existing: Kakao-Compose, Kaspreso Magical

Kotlin DSL

Minimal, syntactic sugar

```
1 sealed class BddKeywords : BddKeywords() {  
2     object Given : BddKeywords()  
3     object When : BddKeywords()  
4     object Then : BddKeywords()  
5     object And : BddKeywords()  
6 }
```

And a series of `infix fun` for `BddKeywords` and `SemanticsNodeInteraction`

Kotlin DSL

```
1  infix fun BddKeywords.node(node: SemanticsNodeInteraction) = node
2
3  infix fun SemanticsNodeInteraction.shouldBe(matcher: SemanticsMatcher) =
4      flakySafely { assert(matcher) }
5
6  infix fun SemanticsNodeInteraction.isDisplayedWith(withProperty: SemanticsI
7      assertIsDisplayed()
8      assert(withProperty)
9      return this
10 }
11
```

Can make the tests

```
1  composeTestRule.onNodeWithText("No Favorites Added").assertIsDisplayed()
2
3  val addButton = composeTestRule.onNodeWithTag("favorite:addButton")
4  addButton.assertIsDisplayed()
5      .assertIsEnabled()
6      .assertHasClickAction()
7      .assert(hasTraversalIndex(1f))
8
9  composeTestRule.onNodeWithTag("favorite:addCurrentButton")
10     .assert(hasTraversalIndex(2f))
```

Can make the tests

```
1 composeTestRule.favoriteScreen {  
2     Then node emptyState shouldBe displayed  
3     And node emptyState shouldBe hasCombinedText("No Favorites Added")  
4     And node addButton isDisplayedWith (enabled  
5         and hasTraversalIndex(1f))  
6     And node addCurrentLocationButton isDisplayedWith (enabled  
7         and hasTraversalIndex(2f))  
8     And node loadingProgress shouldBe notPresent  
9 }
```

Feature File -> Scenario Implementation

```
1 @Favorites @FA-01 @Priority_1 @Android
2   Scenario: FA-01 - Empty state is displayed when no favorite locations are save
3     # Preconditions: @Preconditions_Favorites_EmptyState
4     Given I have no saved favorite locations
5     When I view the Favorites screen
6     Then I see the empty state view with title "No Favorites Added"
7     And I see the "Add Favorite Location" button enabled
8     And I see the "Add Current Location" button enabled
```

Feature File -> Scenario Implementation

```
1  @Scenario("FavoritesScreenFA01: Empty state is displayed when no favorite locat
2  @Priority1
3  fun `empty state and no favorites`() {
4      composeTestRule.favoriteScreen {
5          Then node emptyState shouldBe displayed
6          And node emptyState shouldBe hasCombinedText("No Favorites Added")
7          And node addButton isDisplayedWith (enabled
8              and hasTraversalIndex(1f))
9          And node addCurrentLocationButton isDisplayedWith (enabled
10              and hasTraversalIndex(2f))
11          And node loadingProgress shouldBe notPresent
12      }
13  }
```

Yeah, but... why bother?

- Verbose
- One more layer of abstraction
- **AI tools**

Optimize for what YOU need

If what you care about are:

- shared knowledge
- collective understanding
- easy onboarding
- living documentation of your product

Then prioritize the information for people, rather than machines.

Resources

- <https://developer.android.com/static/develop/ui/compose/images/compose-testing-cheatsheet.pdf>
- <https://roboelectric.org/getting-started>
- <https://github.com/IuliaSTANA/unit-test-ui>



Thank you



Have a great day!

